Dynamic Stack in C++

Generated by Doxygen 1.16.1

1 Dynamic Stack in C++ 1
1.1 Characteristics . . . 1
1.2 Project's structure . . . 1
1.3 Documentation . . . 2
1.4 Execution . . . 2
1.5 Author . . . 2
1.6 Teacher . . . 2
1.7 Subject and Academic program . . . 2
1.8 Date . . . 2

2 Tests 3
2.1 Test results . . . 3

3 Directory Hierarchy 5
3.1 Directories . . . 5

4 Class Index 7
4.1 Class List . . . 7

5 File Index 9
5.1 File List . . . 9

6 Directory Documentation 11
6.1 headers Directory Reference . . . 11
6.2 src Directory Reference . . . 11

7 Class Documentation 13
7.1 Node< T > Class Template Reference . . . 13
7.1.1 Detailed Description . . . 13
7.1.2 Constructor & Destructor Documentation . . . 14
7.1.2.1 Node() [1/3] . . . 14
7.1.2.2 Node() [2/3] . . . 14
7.1.2.3 Node() [3/3] . . . 14
7.1.3 Member Function Documentation . . . 15
7.1.3.1 getNext() . . . 15
7.1.3.2 getValue() . . . 15
7.1.3.3 insertNext() . . . 15
7.1.3.4 insertValue() . . . 16
7.2 Stack< T > Class Template Reference . . . 16
7.2.1 Detailed Description . . . 16
7.2.2 Constructor & Destructor Documentation . . . 17
7.2.2.1 Stack() . . . 17
7.2.2.2 ~Stack() . . . 17
7.2.3 Member Function Documentation . . . 17

7.2.3.1 emptyStack() 17
7.2.3.2 getSize() 18
7.2.3.3 peek() 18
7.2.3.4 pop() 19
7.2.3.5 push() 19

8 File Documentation **21**
8.1 headers/node.h File Reference 21
8.1.1 Detailed Description 21
8.2 node.h 21
8.3 headers/stack.h File Reference 22
8.3.1 Detailed Description 22
8.4 stack.h 22
8.5 README.md File Reference 23
8.6 src/main.cpp File Reference 23
8.6.1 Detailed Description 23
8.6.2 Function Documentation 23
8.6.2.1 main() 23
8.6.2.2 menu() 24
8.7 main.cpp 24
8.8 src/node.cpp File Reference 26
8.8.1 Detailed Description 26
8.9 node.cpp 26
8.10 src/stack.cpp File Reference 27
8.10.1 Detailed Description 27
8.11 stack.cpp 27

Index **29**

Chapter 1

Dynamic Stack in C++

Implementation of a stack in C++ using dynamic memory.

1.1 Characteristics

- Push.
- Pop.
- Peek.
- Automatic Doxygen documentation.

1.2 Project's structure

- build/
  - Unix executable file (exec)
  - dynamStack.dSYM
- doc/
  - html/
  - latex/
- src/
  - stack.cpp
  - node.cpp
  - main.cpp
- headers/
  - stack.h
  - node.h

1.3 Documentation

All the documentation was generated using Doxygen.

To view it: Go to doc/html/index.html and open with your default browser. To check all comments: Click on "Class Members" and then on "All".

Or check the PDF file "refman.pdf".

1.4 Execution

- To run directly: Open and run the Unix executable file (exec).
- By using commands: Open your terminal, and run...

g++ src/main.cpp -Iheaders -o dynamStack ./dynamStack

1.5 Author

Camila Andrea Jaimes Martinez.

1.6 Teacher

Manuel Balderas Victoria.

1.7 Subject and Academic program

- Data structure.
- Software Engineering and Data Mining (1st year).

1.8 Date

March 15, 2026.

Chapter 2

Tests

Execution of the Stack program.

2.1 Test results

The following images show the execution of the Stack program in the terminal, including the functions of Push, Pop, and Peek.

```
Last login: Sun Mar 15 15:07:11 on ttys001
/Users/camila/Desktop/dynam_stack/build/dynamStack ; exit;
camila@MacBook-Air-de-Camila ~ % /Users/camila/Desktop/dynam_stack/build/dynamStack ; exit;


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 11

The node has been added with the value of 11!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 4

The node has been added with the value of 4!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 7

The node has been added with the value of 7!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 33

The node has been added with the value of 33!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.
```

Figure 2.1 Stack execution! (Part 1)

```
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 3

The value stored in the top of the stack is: 33
The number of Nodes in the stack is: 4


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 2

The value stored in the top of the stack is: 33
The top of the stack has been deleted!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 2

The value stored in the top of the stack is: 7
The top of the stack has been deleted!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 3

The value stored in the top of the stack is: 4
The number of Nodes in the stack is: 2


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Exit.

--> Choose an option: 4

Exiting the program...
Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Proceso completado]
```

Figure 2.2 Stack execution! (Part 2)

Chapter 3

Directory Hierarchy

3.1 Directories

headers 11
- node.h 21
- stack.h 22

src 11
- main.cpp 23
- node.cpp 26
- stack.cpp 27

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Node< T > 13
Stack< T > 16

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

headers/node.h
Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next" . 21
headers/stack.h
Defines the structure of the Stack. Its structure consists of a pointer called "top" 22
src/main.cpp
The complete program of the dynamic stack. Includes a menu function so that the user may input any desired value 23
src/node.cpp
Implementation of the constructors of the Node 26
src/stack.cpp
Implementation of the constructors of the stack 27

Chapter 6

Directory Documentation

6.1 headers Directory Reference

Files

- file node.h

  Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next".
- file stack.h

  Defines the structure of the Stack. Its structure consists of a pointer called "top".

6.2 src Directory Reference

Files

- file main.cpp

  The complete program of the dynamic stack. Includes a menu function so that the user may input any desired value.
- file node.cpp

  Implementation of the constructors of the Node.
- file stack.cpp

  Implementation of the constructors of the stack.

Chapter 7

Class Documentation

7.1 Node< T > Class Template Reference

```
#include <node.h>
```

Public Member Functions

- Node (T data, Node< T > *nextNode)

  Initializes the Node with a value and sets the pointer to the next Node.
- Node (T data)

  Initializes the Node with a value and sets its pointer to nullptr.
- Node ()

  Initializes the Node empty (no value nor reference stored).
- void insertValue (T data)

  Inserts a value into the Node.
- void insertNext (Node< T > *nextNode)

  Sets the pointer to the next Node.
- T getValue () const

  Returns the value stored in the Node.
- Node< T > * getNext () const

  Returns the memory address to the next Node.

7.1.1 Detailed Description

template<typename T>
class Node< T >

Definition at line 15 of file node.h.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 Node() [1/3]

```
template<typename T>
Node< T >::Node (
            T data,
            Node< T > * nextNode)
```

Initializes the Node with a value and sets the pointer to the next Node.

Parameters

data	The value to be stored in the Node.
nextNode	The memory address of the next Node.

Definition at line 18 of file node.cpp.

```
00019 {
00020     value = data;
00021     next = nextNode;
00022 }
```

7.1.2.2 Node() [2/3]

```
template<typename T>
Node< T >::Node (
            T data)
```

Initializes the Node with a value and sets its pointer to nullptr.

Parameters

data	The value to be stored in the Node.

Definition at line 33 of file node.cpp.

```
00034 {
00035     value = data;
00036     next = nullptr;
00037 }
```

7.1.2.3 Node() [3/3]

```
template<typename T>
Node< T >::Node ()
```

Initializes the Node empty (no value nor reference stored).

Initializes the Node empty (no value nor reference stored). Default Node constructor.

Definition at line 47 of file node.cpp.

```
00048 {
00049     value = T();
00050     next = nullptr;
00051 }
```

7.1.3 Member Function Documentation

7.1.3.1 getNext()

```
template<typename T>
Node< T > * Node< T >::getNext () const
```

Returns the memory address to the next Node.

Returns

Memory address of the next Node.

Definition at line 104 of file node.cpp.

```
00105 {
00106     return next;
00107 }
```

7.1.3.2 getValue()

```
template<typename T>
T Node< T >::getValue () const
```

Returns the value stored in the Node.

Returns

The value stored in the Node.

Definition at line 90 of file node.cpp.

```
00091 {
00092     return value;
00093 }
```

7.1.3.3 insertNext()

```
template<typename T>
void Node< T >::insertNext (
            Node< T > * nextNode)
```

Sets the pointer to the next Node.

Parameters

nextNode	The memory address of the next Node.

Definition at line 76 of file node.cpp.

```
00077 {
00078     next = nextNode;
00079 }
```

7.1.3.4 insertValue()

```
template<typename T>
void Node< T >::insertValue (
            T data)
```

Inserts a value into the Node.

Parameters

data	The value to be stored.

Definition at line 62 of file node.cpp.

```
00063 {
00064     value = data;
00065 }
```

The documentation for this class was generated from the following files:

- headers/node.h
- src/node.cpp

7.2 Stack< T > Class Template Reference

```
#include <stack.h>
```

Public Member Functions

- Stack ()

  Initializes an empty stack.
- ~Stack ()

  Clears the stack and releases all dynamically allocated resources.
- bool emptyStack () const

  Returns true if the stack has no Nodes, and false otherwise.
- int getSize () const

  Returns the number of Nodes in the stack by traversing the list.
- void push (Node< T > *data)

  Adds a new Node, with a value, to the top of the stack.
- T pop ()

  Deletes the Node associated at the top of the stack.
- T peek () const

  Returns the value of the Node associated at the top of the stack.

7.2.1 Detailed Description

template<typename T>
class Stack< T >

Definition at line 16 of file stack.h.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Stack()

```
template<typename T>
Stack< T >::Stack ()
```

Initializes an empty stack.

Top is set to nullptr, allowing the stack to be empty.

Definition at line 21 of file stack.cpp.

```
00022 {
00023     top = nullptr;
00024 }
```

7.2.2.2 ~Stack()

```
template<typename T>
Stack< T >::~Stack ()
```

Clears the stack and releases all dynamically allocated resources.

Deletes all the Nodes stored in the stack, thus releasing all allocated memory.

Definition at line 35 of file stack.cpp.

```
00036 {
00037     Node<T>* tmp = top;
00038
00039     while (tmp != nullptr)
00040     {
00041         Node<T>* deleteNode = tmp;
00042         tmp = tmp->getNext();
00043         delete deleteNode;
00044     }
00045 }
```

7.2.3 Member Function Documentation

7.2.3.1 emptyStack()

```
template<typename T>
bool Stack< T >::emptyStack () const
```

Returns true if the stack has no Nodes, and false otherwise.

Returns

True if the stack has no Nodes. False if it has at least one Node.

Definition at line 55 of file stack.cpp.

```
00056 {
00057     return top == nullptr;
00058 }
```

7.2.3.2 getSize()

```
template<typename T>
int Stack< T >::getSize () const
```

Returns the number of Nodes in the stack by traversing the list.

Returns

The number of Nodes in the stack.

Definition at line 68 of file stack.cpp.

```
00069 {
00070     int count = 0;
00071     Node<T>* tmp = top;
00072
00073     while (tmp != nullptr)
00074     {
00075         count++;
00076         tmp = tmp->getNext();
00077     }
00078
00079     return count;
00080 }
```

7.2.3.3 peek()

```
template<typename T>
T Stack< T >::peek () const
```

Returns the value of the Node associated at the top of the stack.

///< Returns the value of the Node at the top of the stack.

If the stack is empty, then an error message is thrown. Otherwise, it returns the value inside "top", and in the main file (main.cpp) by using getSize(), it also returns the number of Nodes in the stack. This can be done through the option 3 of the menu.

Returns

The value stored in the top of the stack.

Definition at line 155 of file stack.cpp.

```
00156 {
00157     if (emptyStack())
00158     {
00159         throw runtime_error("The stack has no elements yet!");
00160     }
00161     else
00162     {
00163         return top->getValue();
00164     }
00165 }
```

7.2.3.4 pop()

```
template<typename T>
T Stack< T >::pop ()
```

Deletes the Node associated at the top of the stack.

If the stack is empty, then an error message is thrown. Otherwise, a new pointer "deleteNode" that references to the top of the stack (pointer "top"), while another variable stores the value stored in "top" to return it.

The top of the stack becomes the next Node. Finally, "deleteNode" is deleted.

Returns

The value stored in the deleted Node.

Definition at line 123 of file stack.cpp.

```
00124 {
00125     if (emptyStack())
00126     {
00127         throw runtime_error("The stack has no elements yet!");
00128     }
00129     else
00130     {
00131         Node<T>* deleteNode = top;
00132         T nodeValue = top->getValue();
00133
00134         top = top->getNext();
00135         delete deleteNode;
00136         return nodeValue;
00137     }
00138 }
```

7.2.3.5 push()

```
template<typename T>
void Stack< T >::push (
            Node< T > * newNode)
```

Adds a new Node, with a value, to the top of the stack.

If the stack is empty, the new Node becomes the top of the stack. Otherwise, the pointer of the new Node is set to the memory address of "top", then the new Node becomes the top.

Parameters

newNode	Memory address of the new Node.

Definition at line 95 of file stack.cpp.

```
00096 {
00097     if (emptyStack())
00098     {
00099         top = newNode;
00100     }
00101     else
00102     {
00103         newNode->insertNext(top);
00104         top = newNode;
00105     }
00106 }
```

The documentation for this class was generated from the following files:

- headers/stack.h
- src/stack.cpp

Chapter 8

File Documentation

8.1 headers/node.h File Reference

Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next".

```
#include <string>
#include "../src/node.cpp"
```

Classes

- class Node< T >

8.1.1 Detailed Description

Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next".

A node can store a value of type T in "value", whereas the pointer "next" that references to the next Node in the list. This file defines the constructors of the functions of a Node.

Definition in file node.h.

8.2 node.h

Go to the documentation of this file.

```
#pragma once
#include <string>


template <typename T>
class Node
{
    private:
        T value;
        Node<T>* next;

    public:
        Node(T data, Node<T>* nextNode);
        Node(T data);
        Node();

        void insertValue(T data);
        void insertNext(Node<T>* nextNode);
        T getValue() const;
        Node<T>* getNext() const;
};

#include "../src/node.cpp"
```

8.3 headers/stack.h File Reference

Defines the structure of the Stack. Its structure consists of a pointer called "top".

```
#include <string>
#include "node.h"
#include "../src/stack.cpp"
```

Classes

- class Stack< T >

8.3.1 Detailed Description

Defines the structure of the Stack. Its structure consists of a pointer called "top".

A stack is a linear data structure that follows the LIFO (Last-in, First-out) method, thus making the core of its functions the last element, or in this case Node, added to the stack. This file defines the constructors of the functions of the stack.

Definition in file stack.h.

8.4 stack.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include <string>
00003 #include "node.h"
00004
00014
00015 template <typename T>
00016 class Stack
00017 {
00018     private:
00019
00020         Node<T>* top;
00021
00022     public:
00023
00024         Stack();
00025         ~Stack();
00026
00027         bool emptyStack() const;
00028         int getSize() const;
00029
00030         void push(Node<T>* data);
00031         T pop();
00032
00033         T peek() const;
00034
00035 };
00036
00037 #include "../src/stack.cpp"
```

8.5 README.md File Reference

8.6 src/main.cpp File Reference

The complete program of the dynamic stack. Includes a menu function so that the user may input any desired value.

```
#include <iostream>
#include "node.h"
#include "stack.h"
```

Functions

- int menu ()

  Menu that manages user input.
- int main ()

8.6.1 Detailed Description

The complete program of the dynamic stack. Includes a menu function so that the user may input any desired value.

Option 1 allows the user to input any value of type int. Option 2 allows the user to delete the Node at the top of the stack. Option 3 allows the user to view the value at the top of the stack and its size. Option 4 allows the user to exit the program.

Definition in file main.cpp.

8.6.2 Function Documentation

8.6.2.1 main()

```
int main ()
```

< Initializes a stack of type int.

< Attempts to delete the top of the stack.

< Sends an message if an error occurs.

< Attempts to retrieve the value in top of the stack, as well as the stack's size.

< Sends an message if an error occurs.

Definition at line 19 of file main.cpp.

```
{
    Stack<int> stack;

    while (true)
    {

        switch (menu())
        {
        case 1:
        {
```

```
            int value;
            cout « "\nWhat value is to be stored in the new Node?: ";
            cin » value;

            stack.push(new Node<int>(value));
            cout « "\nThe node has been added with the value of " « value « "!";
            break;
        }
        case 2:
        {
            try
            {
                cout « "\nThe value stored in the top of the stack is: " « stack.pop();
                cout « "\nThe top of the stack has been deleted!";
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 3:
        {
            try
            {
                cout « "\nThe value stored in the top of the stack is: " « stack.peek();
                cout « "\nThe number of Nodes in the stack is: " « stack.getSize();
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 4:
        {
            cout « "\nExiting the program...";
            exit(0);
            break;
        }
        }
    }
}
```

8.6.2.2 menu()

```
int menu ()
```

Menu that manages user input.

Returns an option to the main function.

Definition at line 84 of file main.cpp.

```
{
    int option;
    do
    {
        cout « "\n\n\n\t--- MENU! ---";
        cout « "\n1) Push.\n";
        cout « "2) Pop.\n";
        cout « "3) Peek.\n";
        cout « "4) Exit.\n";
        cout « "\n--> Choose an option: ";
        cin » option;
    } while (option < 1 || option > 4);
    return option;
}
```

8.7 main.cpp

Go to the documentation of this file.

```
#include <iostream>
#include "node.h"
#include "stack.h"


int menu();
using namespace std;

int main()
{
    Stack<int> stack;

    while (true)
    {

        switch (menu())
        {
        case 1:
        {
            int value;
            cout « "\nWhat value is to be stored in the new Node?: ";
            cin » value;

            stack.push(new Node<int>(value));
            cout « "\nThe node has been added with the value of " « value « "!";
            break;
        }
        case 2:
        {
            try
            {
                cout « "\nThe value stored in the top of the stack is: " « stack.pop();
                cout « "\nThe top of the stack has been deleted!";
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 3:
        {
            try
            {
                cout « "\nThe value stored in the top of the stack is: " « stack.peek();
                cout « "\nThe number of Nodes in the stack is: " « stack.getSize();
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 4:
        {
            cout « "\nExiting the program...";
            exit(0);
            break;
        }
        }
    }
}

/*************************************************************************/


int menu()
{
    int option;
    do
    {
        cout « "\n\n\n\t--- MENU! ---";
        cout « "\n1) Push.\n";
        cout « "2) Pop.\n";
        cout « "3) Peek.\n";
        cout « "4) Exit.\n";
        cout « "\n--> Choose an option: ";
        cin » option;
    } while (option < 1 || option > 4);
    return option;
}

/*************************************************************************/

```

8.8 src/node.cpp File Reference

Implementation of the constructors of the Node.

```
#include "node.h"
```

8.8.1 Detailed Description

Implementation of the constructors of the Node.

Definition in file node.cpp.

8.9 node.cpp

Go to the documentation of this file.

```
#include "node.h"


/*************************************************************************/


template <typename T>
Node<T>::Node(T data, Node<T>* nextNode)
{
    value = data;
    next = nextNode;
}

/*************************************************************************/


template <typename T>
Node<T>::Node(T data)
{
    value = data;
    next = nullptr;
}

/*************************************************************************/


template <typename T>
Node<T>::Node()
{
    value = T();
    next = nullptr;
}

/*************************************************************************/


template <typename T>
void Node<T>::insertValue(T data)
{
    value = data;
}

/*************************************************************************/


template <typename T>
void Node<T>::insertNext(Node<T>* nextNode)
{
    next = nextNode;
}

/*************************************************************************/


template <typename T>
```

```
T Node<T>::getValue() const
{
    return value;
}

/*********************************************************************/


template <typename T>
Node<T>* Node<T>::getNext() const
{
    return next;
}
```

8.10 src/stack.cpp File Reference

Implementation of the constructors of the stack.

```
#include <iostream>
#include "stack.h"
#include "node.h"
```

8.10.1 Detailed Description

Implementation of the constructors of the stack.

Definition in file stack.cpp.

8.11 stack.cpp

Go to the documentation of this file.

```
#include <iostream>
#include "stack.h"
#include "node.h"


using namespace std;

/*********************************************************************/


template <typename T>
Stack<T>::Stack()
{
    top = nullptr;
}

/*********************************************************************/


template <typename T>
Stack<T>::~Stack()
{
    Node<T>* tmp = top;

    while (tmp != nullptr)
    {
        Node<T>* deleteNode = tmp;
        tmp = tmp->getNext();
        delete deleteNode;
    }
}

/*********************************************************************/

```

```

template <typename T>
bool Stack<T>::emptyStack() const
{
    return top == nullptr;
}

/*********************************************************************/


template <typename T>
int Stack<T>::getSize() const
{
    int count = 0;
    Node<T>* tmp = top;

    while (tmp != nullptr)
    {
        count++;
        tmp = tmp->getNext();
    }

    return count;
}

/*********************************************************************/


template <typename T>
void Stack<T>::push(Node<T>* newNode)
{
    if (emptyStack())
    {
        top = newNode;
    }
    else
    {
        newNode->insertNext(top);
        top = newNode;
    }
}

/*********************************************************************/


template <typename T>
T Stack<T>::pop()
{
    if (emptyStack())
    {
        throw runtime_error("The stack has no elements yet!");
    }
    else
    {
        Node<T>* deleteNode = top;
        T nodeValue = top->getValue();

        top = top->getNext();
        delete deleteNode;
        return nodeValue;
    }
}


/*********************************************************************/


template <typename T>
T Stack<T>::peek() const
{
    if (emptyStack())
    {
        throw runtime_error("The stack has no elements yet!");
    }
    else
    {
        return top->getValue();
    }
}
```

Index

~Stack
    Stack< T >, 17

Dynamic Stack in C++, 1

emptyStack
    Stack< T >, 17

getNext
    Node< T >, 15
getSize
    Stack< T >, 17
getValue
    Node< T >, 15

headers Directory Reference, 11
headers/node.h, 21
headers/stack.h, 22

insertNext
    Node< T >, 15
insertValue
    Node< T >, 15

main
    main.cpp, 23
main.cpp
    main, 23
    menu, 24
menu
    main.cpp, 24

Node
    Node< T >, 14
Node< T >, 13
    getNext, 15
    getValue, 15
    insertNext, 15
    insertValue, 15
    Node, 14

peek
    Stack< T >, 18
pop
    Stack< T >, 18
push
    Stack< T >, 19

README.md, 23

src Directory Reference, 11
src/main.cpp, 23, 24
src/node.cpp, 26
src/stack.cpp, 27
Stack
    Stack< T >, 17
Stack< T >, 16
    ~Stack, 17
    emptyStack, 17
    getSize, 17
    peek, 18
    pop, 18
    push, 19
    Stack, 17

Tests, 3